Lesson Plan: Module 1

Radio Frequency Module Selection, Hardware Bring-Up, and Beacon–Receiver Subsystem Design

---

An Educational Framework for AI-Driven Signal Processing

Student:	Jude Eschete
Project Advisor:	Dr. Bernard Yett
Program:	M.S. Computer Engineering
Institution:	Stevens Institute of Technology
Date Range:	February 28 – March 14, 2026

This document presents a lesson plan for the first module of the capstone project, covering Radio Frequency transceiver selection, hardware platform validation, and the design of the beacon–receiver subsystem that forms the foundation for all subsequent Field-Programmable Gate Array processing and Machine Learning work.

JEschete@stevens.edu

Contents

1 Module Overview **1**
- 1.1 Context Within the Project . . . 1
- 1.2 Prerequisites . . . 1

2 Weekly Schedule **3**

3 Week 1: RF Module Trade Study **4**
- 3.1 Objectives . . . 4
- 3.2 Selection Criteria . . . 4
- 3.3 Candidate Modules . . . 5
- 3.4 Link Budget Analysis . . . 5

4 Week 2: Hardware Bring-Up and Validation **6**
- 4.1 Objectives . . . 6
- 4.2 STM32 Firmware Bring-Up . . . 6
  - 4.2.1 Development Environment . . . 6
  - 4.2.2 SPI Configuration . . . 7
  - 4.2.3 Validation Steps . . . 8
- 4.3 FPGA Toolchain Validation . . . 8
  - 4.3.1 Development Environment . . . 8
  - 4.3.2 Loopback Design . . . 9

5 Week 3: Beacon Subsystem Design **10**
- 5.1 Objectives . . . 10
- 5.2 Beacon Architecture Overview . . . 10
- 5.3 Packet Format Design . . . 11
- 5.4 Transmission Scheduling . . . 12
  - 5.4.1 Scheduling Modes . . . 12
  - 5.4.2 Timing Implementation . . . 12
- 5.5 Beacon Firmware Architecture . . . 12
- 5.6 Multi-Beacon Validation . . . 13

6 Week 4: Receiver Subsystem Design **15**
- 6.1 Objectives . . . 15
- 6.2 Receiver Architecture . . . 15
- 6.3 Receiver-to-FPGA Interface . . . 15
  - 6.3.1 Interface Selection . . . 15
  - 6.3.2 Receiver-to-FPGA Frame Format . . . 16
- 6.4 Receiver Firmware Architecture . . . 16
- 6.5 FPGA Ingestion Interface . . . 17
- 6.6 System Architecture Diagram . . . 17

7 Hardware Platform Reference **19**

8 Assessment Rubric **19**

9 Recommended Resources **20**
9.1 Datasheets and Technical References 20
9.2 Application Notes 20
9.3 Textbooks 20

10 Transition to Module 2 **21**

Glossary **22**

1 Module Overview

Module 1 Learning Objectives

Upon completion of this module, the student (or future learner following this curriculum) will be able to:

LO-1 Evaluate commercially available Radio Frequency (RF) transceiver modules against project-specific requirements and justify a final selection with quantitative criteria.

LO-2 Verify end-to-end hardware communication between the STMicroelectronics STM32 Nucleo-L476RG microcontroller and the selected RF module using basic firmware.

LO-3 Confirm Field-Programmable Gate Array (FPGA) toolchain readiness on the Digilent Nexys A7-100T and validate a minimal loopback design.

LO-4 Design a complete RF beacon and receiver subsystem architecture, including transmission scheduling, packet formatting, and receiver data flow.

LO-5 Produce a hardware requirements document and subsystem design specification that downstream modules can build upon directly.

1.1 Context Within the Project

This module spans the first two milestones in the project timeline. It establishes the hardware baseline and subsystem architecture upon which all subsequent modules depend. The RF module trade study and hardware bring-up validate that the chosen components can communicate reliably, while the beacon–receiver subsystem design defines the architecture that the FPGA register-transfer level (Register-Transfer Level (RTL)) design (Module 2) and Machine Learning (ML) pipeline (Module 3) will consume.

1.2 Prerequisites

Students beginning this module should have the background listed below. For students who need to catch up on any item, a recommended primer is noted; all cited works appear in full in Section 9.

- Familiarity with C/C++ embedded programming (General-Purpose Input/Output (GPIO), Serial Peripheral Interface (SPI), Universal Asynchronous Receiver/Transmitter (UART), and Inter-Integrated Circuit (I^2C)) — see Valvano, *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers*, for peripheral fundamentals, and the STM32L476xx Reference Manual (RM0351) for the specific peripheral registers used in this module.
- Basic understanding of digital modulation (Amplitude Shift Keying (ASK), Frequency Shift Keying (FSK), and LoRa Chirp Spread Spectrum (CSS) concepts) — Sklar, *Digital Communications: Fundamentals and Applications*, covers the modulation theory; Semtech Application Note AN1200.22, "LoRa Modulation Basics," is the concise primer for LoRa-specific CSS.
- Experience with VHSIC Hardware Description Language (VHDL) and the FPGA synthesis flow — Chu, *FPGA Prototyping by VHDL Examples: Xilinx Artix-7 Edition*, is the primary recommended text; Haskell and Hanna, *Digital Design Using Digilent FPGA Boards: VHDL / Vivado Edition*, provides a Nexys-board-specific walkthrough of the Vivado toolchain.

- Introductory exposure to machine learning (supervised classification) — a formal reference is introduced in Module 4; any standard introductory text on supervised classification is sufficient background here.
- Access to the hardware listed in Section 7

2 Weekly Schedule

The module spans four weeks across two project milestones. Each week has a thematic focus, concrete tasks, and a deliverable checkpoint.

Table 1: Module 1 weekly breakdown.

Week	Focus Area	Key Activities
1	RF Module Trade Study	Define selection criteria; evaluate candidate modules (RFM95W, SX1276, Ra-02, E32); perform link budget analysis; select and procure final module.
2	Hardware Bring-Up	Flash STM32 with SPI driver firmware; verify RF module register access; transmit/receive basic packets; confirm Nexys A7 toolchain and loopback; produce hardware requirements document.
3	Beacon Subsystem Design	Define beacon transmission architecture; design packet format and framing protocol; implement configurable transmission scheduling on STM32; validate multi-beacon operation.
4	Receiver Subsystem Design	Design receiver data path from RF front-end to FPGA input; define FPGA ingestion interface; implement receiver firmware; validate end-to-end beacon-to-FPGA data flow.

3 Week 1: RF Module Trade Study

3.1 Objectives

- Define quantitative and qualitative selection criteria for the RF transceiver module.
- Evaluate at least four candidate modules against those criteria.
- Perform a basic link budget analysis for the benchtop operating environment.
- Produce a justified final selection with procurement plan.

3.2 Selection Criteria

The following table defines the evaluation dimensions. Each criterion is weighted to reflect its importance to the project's educational and technical goals.

Table 2: RF module selection criteria and weights.

Criterion	Weight	Description
SPI Interface Compatibility	20%	Must support standard SPI (CPOL/CPHA modes) at voltages compatible with the STM32 Nucleo (3.3 V logic).
Frequency Band	15%	915 MHz industrial, scientific, and medical (ISM) band preferred for U.S. regulatory compliance.
Modulation Support	15%	Must support LoRa (CSS) and ideally FSK/on-off keying (OOK) for multi-modulation classification exercises.
Documentation & Community	15%	Availability of datasheets, application notes, open-source driver libraries, and community forums.
Cost & Availability	10%	Per-unit cost and lead time; must be procurable from standard distributors (Digi-Key, Mouser, Adafruit).
Power Consumption	10%	Low enough for USB-powered benchtop operation; transmit current < 120 mA.
Packet Engine Features	10%	On-chip first in, first out (FIFO) buffering, cyclic redundancy check (CRC), and packet handling to simplify initial firmware development.
Physical Form Factor	5%	Breadboard- or breakout-board-friendly; no fine-pitch ball grid array (BGA) requiring reflow.

3.3 Candidate Modules

Table 3: Candidate RF transceiver modules.

Module	Chipset	Freq.	Notes
Adafruit RFM95W	SX1276	915 MHz	Breadboard-ready breakout; extensive tutorials.
AI-Thinker Ra-02	SX1278	433 MHz	Low cost; 433 MHz limits U.S. ISM exercises.
EByte E32-915T20D	SX1276	915 MHz	UART interface (not SPI); 100 mW TX power.
HopeRF RFM96W	SX1276	915 MHz	SMD module; needs custom breakout or adapter.

3.4 Link Budget Analysis

For the benchtop environment (transmitter–receiver separation $\leq 10\,\mathrm{m}$ indoors), students should compute:

$$P_{\mathrm{rx}} = P_{\mathrm{tx}} + G_{\mathrm{tx}} + G_{\mathrm{rx}} - L_{\mathrm{path}} - L_{\mathrm{misc}} \tag{1}$$

where:

- P_{tx}: Transmit power (dBm), typically +13 to +20 dBm for SX1276-based modules.
- $G_{\mathrm{tx}}, G_{\mathrm{rx}}$: Antenna gains (dBi); assume $\approx 2\,\mathrm{dBi}$ for standard whip antennas.
- L_{path}: Free-space path loss at 915 MHz over distance d:

$$L_{\mathrm{path}} = 20\log_{10}(d) + 20\log_{10}(f) - 147.55 \quad [\mathrm{dB}] \tag{2}$$

- L_{misc}: Miscellaneous losses (cable, mismatch, indoor multipath margin $\approx 10\,\mathrm{dB}$).

Exercise: Link Budget Worksheet

Calculate the received power for the following scenario:

- $P_{\mathrm{tx}} = +14\,\mathrm{dBm}$, $G_{\mathrm{tx}} = G_{\mathrm{rx}} = 2\,\mathrm{dBi}$
- $d = 5\,\mathrm{m}$, $f = 915\,\mathrm{MHz}$
- $L_{\mathrm{misc}} = 10\,\mathrm{dB}$ (indoor margin)

Verify that P_{rx} exceeds the SX1276 receiver sensitivity ($-137\,\mathrm{dBm}$ at SF12, 125 kHz BW) by a comfortable margin. Document the link margin and discuss its implications for multi-beacon scaling.

Week 1 Deliverable

RF Module Trade Study Report — A 3–5 page document containing the weighted evaluation matrix, link budget calculations, final module selection with justification, and a procurement plan (vendor, cost, estimated delivery).

4 Week 2: Hardware Bring-Up and Validation

4.1 Objectives

- Establish SPI communication between the STM32 Nucleo-L476RG and the selected RF module.
- Transmit and receive basic test packets between two beacon nodes.
- Validate the Nexys A7-100T FPGA toolchain (Vivado) with a minimal loopback design.
- Document all hardware interfaces, pin assignments, and power requirements.

4.2 STM32 Firmware Bring-Up

4.2.1 Development Environment

Two supported workflows are documented. Students may choose either; the course reference environment is the VS Code flow because it unifies the STM32, serial monitor, and (later) documentation/version-control workflow in a single tool.

Option A — VS Code with the official STMicroelectronics extension stack (recommended). Install the *STM32 VS Code Extension* from the Visual Studio Code marketplace. It is the official ST-maintained port of the STM32CubeIDE toolchain and automatically installs the supporting sub-extensions listed below. No separate CubeIDE installation is required.

- **Umbrella extension:** `stmicroelectronics.stm32-vscode-extension` (pulls the full sub-extension set).
- **Project management and build:** `stm32cube-ide-project-manager`, `stm32cube-ide-build-cmake`, `stm32cube-ide-build-analyzer`, and `stm32cube-ide-bundles-manager` (auto-fetches the ARM GCC toolchain and Hardware Abstraction Layer (HAL) packs).
- **IntelliSense:** `stm32cube-ide-clangd` (pre-configured for ARM cross-compilation).
- **Debug:** `stm32cube-ide-debug-core` with the `debug-stlink-gdbserver` back end (the Nucleo-L476RG's on-board ST-LINK/V2.1 is supported out of the box; `debug-jlink-gdbserver` and `debug-generic-gdbserver` are also installed for external probes).
- **Peripheral register view:** `stm32cube-ide-registers` for live register inspection during debug.
- **RTOS awareness:** `stm32cube-ide-rtos` (unused for the Module 1 bare-metal firmware; retained for later modules if an RTOS is introduced).

- **Supporting general extensions:**
  - `ms-vscode.cpptools` (+ extension pack) — fallback C/C++ language server.
  - `ms-vscode.cmake-tools` — CMake integration.
  - `marus25.cortex-debug` — alternate Cortex-M debugger with SVD, RTT, and ITM support.
  - `dan-c-underwood.arm` — ARM assembly syntax highlighting.
  - `eclipse-cdt.serial-monitor` — UART monitor for the Nucleo's virtual COM port directly inside VS Code.

With this stack installed, the full workflow — peripheral configuration (integrated CubeMX), code generation, build, flash over the on-board ST-LINK, single-step debug, peripheral register inspection, and UART monitoring — runs entirely inside VS Code with no additional tools required.

Option B — Standalone STM32CubeIDE. The classic Eclipse-based STM32CubeIDE (free from ST) remains fully supported. It bundles the HAL libraries and STM32CubeMX code generation in a single installer and may be preferred by students already familiar with Eclipse or those who want CubeMX as a dedicated GUI application.

Optional — STM32Cube Command Line Tools (STM32Cube) (CLT). The STM32Cube Command Line Tools package is a single installer that places the ARM GCC toolchain (`arm-none-eabi-gcc`, `-gdb`, `-objcopy`, . . .), CMake, Ninja, `OpenOCD`, the ST-LINK GDB server, and `STM32_Programmer_CLI` on the system `PATH`. It is not required for the in-lab flow — both options above handle build, flash, and debug entirely through their own integrated tooling — but installing it is recommended if you intend to (a) script builds or flashing from a shell or CI pipeline, (b) run `arm-none-eabi-gdb` or `OpenOCD` outside of VS Code, or (c) keep a single toolchain install that survives Integrated Development Environment (IDE) upgrades. The official STM32 VS Code extension auto-discovers a CLT installation when present and prefers it over the Bundles Manager download.

Common target details (all options).

- **Target Microcontroller Unit (MCU):** STM32L476RG (ARM Cortex-M4, 80 MHz, low-power).
- **Debug Interface:** On-board ST-LINK/V2.1 (USB) — no external programmer required.
- **Serial console:** Nucleo virtual COM port via the same ST-LINK USB connection (used for debug prints and Received Signal Strength Indicator (RSSI)/Signal-to-Noise Ratio (SNR) logging).

4.2.2 SPI Configuration

Using the integrated STM32CubeMX peripheral configurator (available from either the VS Code extension or standalone STM32CubeIDE — see Section 4.2.1), configure the SPI peripheral as follows:

Table 4: SPI peripheral configuration for RF module communication.

Parameter	Setting	Rationale
Mode	Full-Duplex Master	MCU drives clock to RF module
Clock Polarity (CPOL)	0	SX1276 default
Clock Phase (CPHA)	0	SX1276 default
Data Size	8 bits	Register-level access
Baud Rate	$\leq 10\,\text{MHz}$	SX1276 SPI max = 10 MHz
NSS	Software (GPIO)	Manual chip select control

4.2.3 Validation Steps

1. **Register Read Test:** Read the SX1276 `RegVersion` register (address `0x42`). Expected return value: `0x12`. This confirms SPI wiring and clock configuration.
2. **Register Write/Read-Back:** Write a known value to a writable register (e.g., `RegFrfMsb`), read it back, and verify.
3. **Mode Transition:** Command the module from `SLEEP` → `STANDBY` → `TX` and verify the mode bits in `RegOpMode`.
4. **Packet Transmission:** Configure two STM32+RF nodes. Transmit a known payload from Node A; verify reception and CRC pass on Node B. Log RSSI and SNR values.

Common Pitfalls

- **Level mismatch:** Ensure the RF module operates at 3.3 V logic. The Nucleo's Arduino-compatible headers provide 3.3 V by default, but verify with a multimeter before connecting.
- **SPI bus contention:** If other SPI peripherals share the bus, ensure proper chip-select management.
- **Antenna:** Always connect an antenna (or 50 Ω dummy load) before transmitting to avoid damage to the RF front-end.
- **Reset sequencing:** The SX1276 requires a hardware reset pulse (active low, $\geq 100\,\mu$s) after power-on before SPI access is reliable.

4.3 FPGA Toolchain Validation

4.3.1 Development Environment

- **Tool:** AMD Vivado Design Suite (WebPACK edition, free for Artix-7).
- **Board:** Digilent Nexys A7-100T (XC7A100T-1CSG324C).
- **Constraint File:** Use the Digilent-provided `Nexys-A7-100T-Master.xdc`.

4.3.2 Loopback Design

Implement a minimal design to validate the toolchain and board:

1. **LED Echo:** Map the 16 slide switches to the 16 LEDs (combinational pass-through). Confirms pin constraints and bitstream generation.
2. **UART Loopback:** Instantiate a simple UART receiver and transmitter at 115200 baud on the USB-UART bridge. Echo received characters back to the host terminal. Confirms clocking, UART IP, and host communication.
3. **Clock Verification:** Use an Integrated Logic Analyzer (ILA) (Integrated Logic Analyzer) core to probe a free-running counter driven by the 100 MHz oscillator. Verify the expected count rate.

Week 2 Deliverable

Hardware Requirements Document — A formal document capturing:

1. **Bill of Materials (Bill of Materials (BOM)):** Part numbers, quantities, unit costs, vendors, and lead times for all components.
2. **Interface Specification:** Pin mappings for STM32↔RF module (SPI + GPIO), planned FPGA↔receiver interface, UART debug connections.
3. **Power Budget:** Measured current draw for each subsystem; confirm USB power sufficiency.
4. **Validation Evidence:** Screenshots or logic analyzer captures of successful SPI transactions, packet reception logs with RSSI/SNR, FPGA loopback test results.
5. **Risk Register:** Identified risks (e.g., RF interference in lab environment, component lead times) with mitigation strategies.

5 Week 3: Beacon Subsystem Design

5.1 Objectives

- Define the role and behavior of each beacon node within the system.
- Design a packet format that carries the metadata required for downstream classification.
- Implement configurable transmission scheduling that supports multiple simultaneous beacons.
- Validate multi-beacon operation and characterize the RF channel under concurrent transmissions.

5.2 Beacon Architecture Overview

Each beacon consists of an STM32 Nucleo-L476RG paired with the selected LoRa transceiver module. Beacons operate as cooperative targets: they transmit known signals at configurable intervals and parameters so that the receiver and FPGA processing chain have a controlled, characterizable input. The beacon subsystem must be designed so that students can scale from a single beacon to N simultaneous beacons, progressively increasing the demands on the receiver and FPGA.

Design Principle: Controlled Complexity

The beacon subsystem is intentionally designed to give students direct control over the RF environment. By parameterizing transmission power, modulation type, spreading factor, and timing, students can systematically vary the input to the FPGA and ML pipeline, creating a structured experimental platform rather than relying on uncontrolled ambient signals.

5.3 Packet Format Design

Each beacon transmission must carry sufficient metadata for the receiver and FPGA to identify, classify, and prioritize it. The following packet structure is recommended:

Table 5: Beacon packet format (fixed-length, 32 bytes).

Field	Offset	Size	Description
Preamble Sync	0	2 B	Fixed sync word (`0xAE5A`) for frame detection.
Beacon ID	2	1 B	Unique identifier for this beacon (0x01–0xFF).
Sequence Number	3	2 B	Monotonically increasing counter; enables packet loss detection.
TX Power	5	1 B	Transmit power setting (dBm), encoded as unsigned integer.
Modulation Code	6	1 B	Identifies modulation type: 0x01=LoRa, 0x02=FSK, 0x03=OOK.
Spreading Factor	7	1 B	LoRa SF value (7–12); 0x00 if non-LoRa modulation.
Bandwidth Code	8	1 B	Encoded BW setting (e.g., 0x00=125 kHz, 0x01=250 kHz).
Timestamp	9	4 B	STM32 system tick (ms) at time of transmission.
Payload	13	17 B	User-defined data or pseudo-random training sequence.
CRC-16	30	2 B	CRC over bytes 0–29 for integrity verification.

Design Rationale: Why These Fields?

The metadata fields (TX Power, Modulation Code, SF, BW) serve a dual purpose. First, they provide ground-truth labels for the ML classification pipeline in later modules — the receiver knows what the beacon *claims* it transmitted, enabling supervised training. Second, they allow the FPGA processing chain to verify its own detection and identification logic against known parameters.

The 17-byte payload region can carry pseudo-random bit sequences for bit-error-rate (BER) testing or structured data patterns that exercise specific aspects of the demodulator.

5.4 Transmission Scheduling

When multiple beacons share the same frequency band, uncoordinated transmissions will collide. The scheduling strategy must balance simplicity (suitable for an educational context) with sufficient sophistication to create realistic multi-access scenarios.

5.4.1 Scheduling Modes

Students should implement at least two scheduling modes:

1. **Fixed Interval (TDMA-like):** Each beacon is assigned a time slot within a repeating frame. Slot assignments are configured at compile time or via UART command. This mode produces predictable, non-colliding traffic and is the starting point for initial receiver validation.

$$t_{\text{tx}}^{(k)} = T_{\text{frame}} \cdot n + \Delta t \cdot k, \quad k = 0, 1, \ldots, N-1 \tag{3}$$

   where T_{frame} is the frame period, Δt is the slot duration, k is the beacon index, and n is the frame counter.

2. **Randomized Interval (ALOHA-like):** Each beacon transmits after a random back-off drawn from a configurable uniform distribution $[T_{\min}, T_{\max}]$. This mode introduces collisions and partial overlaps, creating a more challenging scenario for the FPGA detection and the ML classifier.

5.4.2 Timing Implementation

Transmission timing is driven by the STM32's hardware timer peripheral (TIM2 or TIM6) configured in one-shot or periodic mode. Key implementation details:

- Use the STM32 HAL timer interrupt callback (`HAL_TIM_PeriodElapsedCallback`) to trigger transmissions with minimal jitter.
- For randomized mode, seed the pseudo-random number generator with a unique value per beacon (e.g., beacon ID XOR'd with an ADC noise sample).
- Log each transmission timestamp over UART for ground-truth correlation with receiver data.

5.5 Beacon Firmware Architecture

The beacon firmware should be structured as a simple state machine:

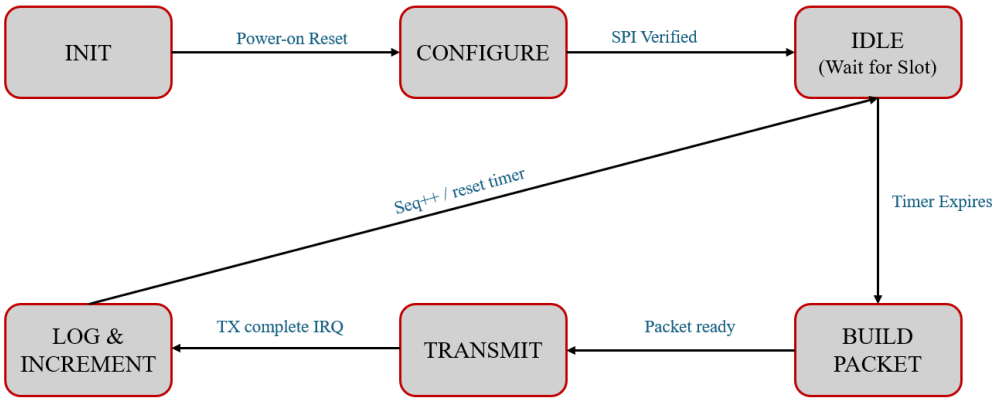

Figure 1: Beacon firmware state machine. The IDLE→BUILD→TRANSMIT→LOG cycle repeats for each scheduled transmission.

5.6 Multi-Beacon Validation

Exercise: Two-Beacon Concurrent Operation

Configure two beacons with the following parameters and verify correct operation:

Parameter	Beacon A	Beacon B
Beacon ID	`0x01`	`0x02`
TX Power	+14 dBm	+10 dBm
Spreading Factor	SF7	SF10
Scheduling Mode	Fixed, 2 s interval	Fixed, 3 s interval

Using a third STM32+RF module as a simple packet sniffer, capture 60 seconds of traffic and verify:

1. Both beacon IDs appear in received packets.
2. Sequence numbers are monotonically increasing for each beacon.
3. RSSI values differ between the two beacons (reflecting the TX power difference).
4. No CRC failures occur under the fixed-interval schedule.

Week 3 Deliverable

Beacon Subsystem Design Document — A document containing:

1. Packet format specification with field descriptions and byte layout diagram.
2. Transmission scheduling design (both TDMA and ALOHA modes) with timing equations.
3. Beacon firmware architecture (state machine diagram and module descriptions).
4. Multi-beacon validation test results (packet captures, RSSI logs, sequence verification).

6 Week 4: Receiver Subsystem Design

6.1 Objectives

- Design the receiver data path from the RF module through the STM32 to the FPGA.
- Define the interface protocol between the receiver MCU and the Nexys A7 FPGA.
- Implement receiver firmware that captures, timestamps, and forwards received packets.
- Validate end-to-end data flow from beacon transmission to FPGA ingestion.

6.2 Receiver Architecture

The receiver station uses a dedicated STM32 Nucleo-L476RG paired with an RF module configured in continuous receive mode. Unlike the beacons, the receiver must handle packets from multiple asynchronous sources. Its primary responsibilities are:

1. **Packet Reception:** Detect and demodulate incoming LoRa/FSK packets using the SX1276's internal packet engine and DIO interrupt lines.
2. **Metadata Extraction:** Read hardware-reported RSSI, SNR, and frequency error from the SX1276 registers for each received packet.
3. **Timestamping:** Capture the STM32 system tick at the moment the `RxDone` interrupt fires, providing a receiver-side timestamp independent of the beacon's claimed timestamp.
4. **Forwarding:** Package the received payload and metadata into a structured frame and transmit it to the FPGA over the chosen interface (SPI or UART).

6.3 Receiver-to-FPGA Interface

6.3.1 Interface Selection

Table 6: Interface options for receiver–FPGA communication.

Interface	Throughput	Complexity	Trade-offs
UART	≤ 921.6 kbps	Low	Simple two-wire connection; sufficient for low beacon counts; well-supported in both STM32 HAL and VHDL.
SPI (MCU as master)	≤ 10 Mbps	Medium	Higher throughput for scaling; requires clock, MOSI, MISO, and CS lines; FPGA acts as SPI slave.

Recommendation

Begin with UART at 115200 baud for initial integration. At 32 bytes per packet with framing overhead (∼40 bytes total), UART supports approximately 28 packets per second, which is sufficient for the initial two-beacon configuration. Transition to SPI when scaling beyond 4–5 beacons or when latency requirements tighten in later modules.

6.3.2 Receiver-to-FPGA Frame Format

The receiver wraps each received packet with additional metadata before forwarding to the FPGA:

Table 7: Receiver-to-FPGA forwarding frame (43 bytes).

Field	Offset	Size	Description
Start Delimiter	0	1 B	Fixed byte `0x7E` for frame synchronization.
Frame Length	1	1 B	Total frame length (excluding start delimiter and CRC).
RX Timestamp	2	4 B	Receiver-side system tick (ms) at `RxDone` interrupt.
RSSI	6	1 B	Received signal strength (dBm, two's complement).
SNR	7	1 B	Signal-to-noise ratio (dB, signed, 0.25 dB resolution).
Freq. Error	8	2 B	Frequency error estimate from SX1276 (Hz, signed 16-bit).
Beacon Payload	10	32 B	Complete beacon packet as received (Table 5).
Frame CRC-8	42	1 B	CRC over bytes 1–41 for transport integrity.

6.4 Receiver Firmware Architecture

The receiver firmware operates in an interrupt-driven architecture:

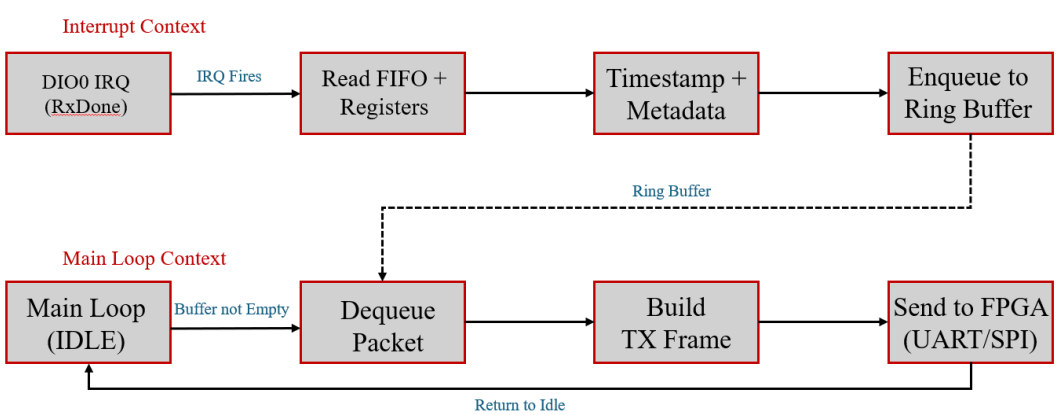

Figure 2: Receiver firmware architecture. The ISR captures packets into a ring buffer; the main loop drains the buffer and forwards frames to the FPGA.

Concurrency Considerations

- The ring buffer must be sized to handle burst arrivals. With two beacons at 1-second intervals and a 43-byte frame, a 512-byte buffer (11 frames) provides ample margin.
- Use `volatile` qualifiers and disable/enable interrupts around shared index variables to prevent race conditions between the ISR and main loop.
- If the main loop cannot drain the buffer fast enough (detectable by a buffer-full flag), log an overflow event — this becomes a design constraint that informs FPGA throughput requirements in Module 2.

6.5 FPGA Ingestion Interface

On the FPGA side, the receiver data is captured by a simple UART receiver module (or SPI slave, if that interface is selected). The Week 4 scope is limited to:

1. **UART Receiver IP:** Implement or instantiate a UART RX module on the Nexys A7 that receives bytes at the configured baud rate and writes them to a small FIFO.
2. **Frame Detection:** Implement start-delimiter detection (`0x7E`) and frame-length extraction to identify complete frames in the byte stream.
3. **Verification:** Display received beacon IDs on the Nexys A7 seven-segment display or LEDs, confirming end-to-end data flow from beacon through receiver to FPGA fabric.

Exercise: End-to-End Data Flow Validation

With two beacons transmitting in fixed-interval mode:

1. Verify that the FPGA receives frames from both beacons by displaying alternating beacon IDs on the seven-segment display.
2. Compare the receiver-side RSSI values displayed on the FPGA with the values logged by the STM32 over UART debug to confirm data integrity across the interface.
3. Introduce a deliberate CRC error (corrupt one byte in the receiver firmware before forwarding) and confirm the FPGA's frame CRC check rejects the corrupted frame.

6.6 System Architecture Diagram

Figure 3 shows the complete subsystem architecture validated by the end of this module.

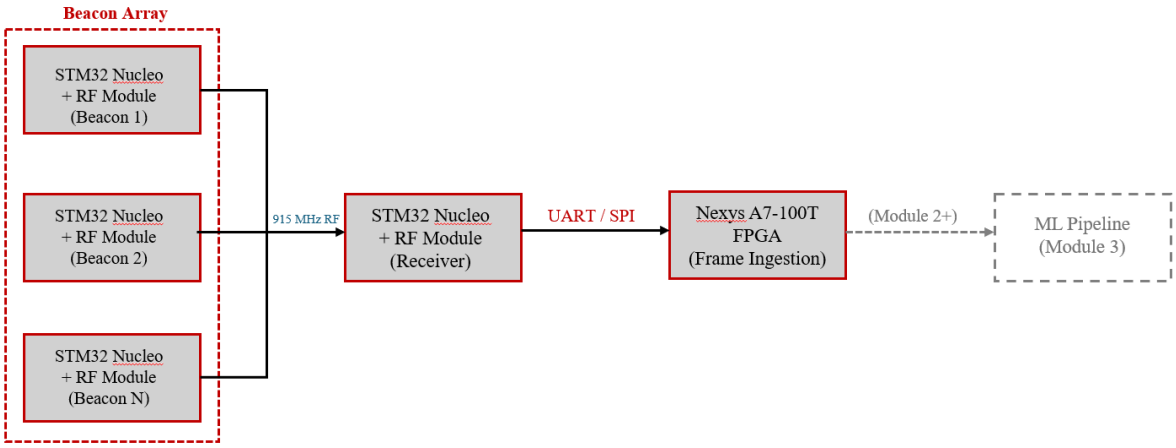

Figure 3: Module 1 subsystem architecture. Solid blocks and arrows indicate components and data paths validated by end of Week 4. The dashed ML Pipeline block indicates a downstream module.

Week 4 Deliverable

Receiver Subsystem Design Document — A document containing:

1. Receiver architecture description and data flow diagram.
2. Receiver-to-FPGA interface selection rationale and frame format specification.
3. Receiver firmware design (interrupt architecture, ring buffer sizing, forwarding logic).
4. FPGA ingestion module description (UART RX, frame detection, verification outputs).
5. End-to-end validation test results (beacon→receiver→FPGA data flow confirmation).

7 Hardware Platform Reference

Table 8: Required hardware components for Module 1.

Component	Qty	Purpose
STM32 Nucleo-L476RG	≥ 3	Two beacon MCUs + one receiver MCU
RF Transceiver Module	≥ 3	Sub-GHz LoRa communication (e.g., RFM95W, 915 MHz)
915 MHz Whip Antenna	≥ 3	Required for RF transmission/reception
Digilent Nexys A7-100T	1	FPGA development board (Artix-7, 100T)
USB Micro-B Cables	4–5	Power and programming for Nucleo and Nexys boards
Breadboard + Jumper Wires	—	Prototyping connections (SPI, GPIO, power, UART)
Logic Analyzer (optional)	1	SPI/UART bus debugging and timing verification

8 Assessment Rubric

Table 9: Module 1 assessment rubric.

Assessment Item	Weight	Due
RF Module Trade Study Report (weighted matrix, link budget, justification)	20%	Week 1
Hardware Requirements Document (BOM, interfaces, power budget, validation evidence)	20%	Week 2
Beacon Subsystem Design (packet format, scheduling, firmware, multi-beacon tests)	30%	Week 3
Receiver Subsystem Design (data path, interface, firmware, end-to-end validation)	30%	Week 4

9 Recommended Resources

9.1 Datasheets and Technical References

1. Semtech, "SX1276/77/78/79 Datasheet," Rev. 7, 2020.
2. STMicroelectronics, "STM32L476xx Reference Manual," RM0351, Rev. 9, 2023.
3. Xilinx/AMD, "Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181, 2023.
4. Digilent, "Nexys A7 Reference Manual," Rev. C, 2022.

9.2 Application Notes

1. Semtech AN1200.22, "LoRa Modulation Basics."
2. Semtech AN1200.13, "SX1276/7/8 LoRa Modem Design Guide."
3. STMicroelectronics AN4286, "SPI Protocol Used in the STM32 Bootloader."
4. STMicroelectronics AN4013, "STM32 Timer Overview and Low-Power Timer Usage."

9.3 Textbooks

1. P. P. Chu, *FPGA Prototyping by VHDL Examples: Xilinx Artix-7 Edition*, 2nd ed., Wiley, 2017.
2. R. E. Haskell and D. M. Hanna, *Digital Design Using Digilent FPGA Boards: VHDL / Vivado Edition*, LBE Books, 2019.
3. C. Ünsalan and B. Tar, *Digital System Design with FPGA: Implementation Using Verilog and VHDL*, McGraw-Hill, 2017.
4. B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed., Prentice Hall, 2001.
5. J. W. Valvano, *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers*, 6th ed., 2023.

10 Transition to Module 2

Upon successful completion of Module 1, students will have:

- A validated hardware platform with confirmed SPI communication across all nodes.
- A fully specified and tested beacon subsystem capable of multi-beacon concurrent operation.
- A receiver subsystem that captures, timestamps, and forwards packets to the FPGA.
- End-to-end data flow from beacon transmission through receiver to FPGA frame ingestion.
- Comprehensive design documentation covering packet formats, interfaces, and firmware architectures.

Module 2 will build directly on these foundations with FPGA RTL design and signal processing simulation. The UART/SPI ingestion interface validated in Week 4 becomes the input to the FPGA processing pipeline, and the beacon packet format defines the data structures that the RTL must parse, classify, and prioritize in real time.

Glossary

ASK	Amplitude Shift Keying	1
BOM	Bill of Materials	9, 19
CLT	Command Line Tools (STM32Cube)	7
CRC	Cyclic Redundancy Check	4, 8, 11, 13, 16, 17
CSS	Chirp Spread Spectrum	1, 4
FIFO	First In, First Out	4, 17
FPGA	Field-Programmable Gate Array	1, 3, 6, 9, 10, 11, 12, 15, 16, 17, 18, 19, 20, 21
FSK	Frequency Shift Keying	1, 4, 11, 15
GPIO	General-Purpose Input/Output	1, 8, 9, 19
HAL	Hardware Abstraction Layer	6, 7, 12, 15
I^2C	Inter-Integrated Circuit	1
IDE	Integrated Development Environment	7
ILA	Integrated Logic Analyzer	9
MCU	Microcontroller Unit	7, 8, 15, 19
ML	Machine Learning	1, 10, 11, 12, 18
OOK	On-Off Keying	4, 11
RF	Radio Frequency	1, 3, 4, 5, 6, 8, 9, 10, 13, 15, 19
RSSI	Received Signal Strength Indicator	7, 8, 9, 13, 14, 15, 16, 17
RTL	Register-Transfer Level	1, 21
SNR	Signal-to-Noise Ratio	7, 8, 9, 15, 16
SPI	Serial Peripheral Interface	1, 3, 4, 5, 6, 7, 8, 9, 15, 16, 17, 19, 20, 21
UART	Universal Asynchronous Receiver/Transmitter	1, 5, 9, 12, 15, 16, 17, 18, 19, 21
VHDL	VHSIC Hardware Description Language	1, 15, 20

End of Module 1 Lesson Plan